Parallax — Parallel VHDL Simulator

Date: 27 Feb 2026

NAME: CHIRAG KATHPALIA, **ENTRY NUMBER**: 2025MCS2098

This document is: A design document for Parallax, a parallel VHDL event-driven simulator built as part of COD7001. This covers what I am going to be building, why, how we plan to build it, and what correctness even means in this context.

Link to Github repo:
https://github.com/KathpaliaChirag/Parallax—A-Parallel-VHDL-Simulator

---

The problem in short is

Traditional VHDL simulators are sequential...they process one event at a time, in time order, on a single core. Modern machines have 8, 16, 32 cores sitting completely idle. Parallax is a try to fix this.

But why is this problem hard?

Now we generally think...just throw threads at it. Run events in parallel...maybe use OMP or something and its Done...

Except no. Events have dependencies. If signal A feeds into process B which feeds into signal C, we cannot simply run those in parallel as in say B needs A to finish first. And then there is the concept of delta cycles (I do not understand it completely yet) however VHDL's way of handling zero-time cascading updates makes the ordering problem even more subtle...

So the real problem as i see and understand is: **how do you find which events are safe to run in parallel, run those in parallel, and still get bit-for-bit identical output compared to the sequential version?**

That is what my Parallax is going to be solving.

How Much of VHDL I am actually going to work with

Now after spending a long day on reading about it what I understand is full VHDL is massively enormous... I am not building a full compiler... time is the constraint here...My Parallax will support a synthesizable subset that is enough to simulate real circuits:

What I am planning to make work (initially atleast):

- `entity` and `architecture` declarations
- `signal` declarations
- `process` with sensitivity lists
- Signal assignments with `after` delays
- Basic logic: `and`, `or`, `not`, `xor`

- Basic `if` statements inside processes

What may not work yet (may add later if time permits):

- `generate` statements
- `component` instantiation
- `function` and `procedure`
- `generic` maps

This is not me being lazy...this subset handles logic gates, flip flops, FSMs, and basic pipelines which is exactly what the benchmarks need or atleast thats what i understand...although if i am allowed to use a parser of vhdl that would help in cutting off some time and add more things...but if i have to write a parser of my own...I mean naaahh I can't commit that much... Everything else may get added incrementally once the core works.

File structure and why it is designed this way

here i took help of AI as I do not completely understand this myself but after reading about it I feel this is what i can see working for now (I may change it if needed but change will be minimal)

```
Parallax/
|-- src/
|   |-- parser/          # Flex/Bison...takes VHDL in, spits AST
   out
|   |   |-- lexer.l
|   |   |-- parser.y
|   |   |-- ast.h
|   |   `-- ast.c
|   |-- core/            # heart of the simulator
|   |   |-- event.h/c        # what an event looks like (done
   atleast base)
|   |   |-- event_queue.h/c  # min-heap priority queue (done
   atleast base)
|   |   |-- signal.h/c       # signal state + history
|   |   |-- process.h/c      # process + sensitivity list
|   |   |-- delta.h/c        # delta cycle logic
|   |   `-- scheduler.h/c    # ties core together
|   |-- sim/             # two simulators, same core
|   |   |-- sequential.h/c   # baseline...correct but slow
|   |   `-- parallel.h/c     # the actual contribution
|   |-- analysis/        # dependency graph builder
|   |   |-- dependency.h/c
|   |   `-- graph.h/c
|   |-- sync/            # parallel-specific synchronization
|   |   |-- barrier.h/c
|   |   `-- lockfree.h/c
|   `-- output/
```

```
|       |-- vcd.h/c          # waveform output...viewable in
   GTKWave
|       `-- trace.h/c        # trace hashing for correctness
   checking
|-- tests/
|   |-- unit/            # test each module independently
|   |-- circuits/        # actual VHDL test files
|   |   |-- basic/       # gates, simple logic
|   |   |-- fsm/         # finite state machines
|   |   `-- pipeline/    # pipeline circuits
|   `-- benchmarks/      # speedup measurement
|-- docs/
|-- Makefile
`-- main.c
```

The idea of design decision here is that `core/` is shared between sequential and parallel... Both simulators use the same event queue, same signal representation, same delta cycle logic. The only difference is how processes get scheduled. This means if sequential is correct, parallel just needs to prove it produces the same output... I hope that should be enough.

How we find parallelism...Dependency Analysis

The idea is going to be very simple... Two processes are independent if they do not share signals...one does not read what the other writes. We build a static dependency graph from the AST:

- **Nodes** = processes
- **Edges** = shared signals (read/write conflict)

Processes with no edge between them at the same timestamp are safe to run in parallel.

Delta cycle boundaries are natural synchronization points... No process crosses a delta boundary until every process in the current delta is done. This is enforced with barriers.

Parallelization strategy...Time-slice parallelism

For this too I took help of internet and AI tools... so what i understand is at each simulation timestamp, we collect all events due at that time, build the dependency graph for that slice, and run independent processes in parallel using OpenMP. Dependent processes are serialized.

```
while queue not empty:
    collect all events at current_time
    build dependency graph for this slice
    assign independent processes to threads (OpenMP)
    barrier...wait for all threads to finish
    resolve delta cycles
    advance simulation time
```

We chose time-slice parallelism over speculative execution because it is simpler to prove correct and has no rollback overhead. Speculative execution would give more parallelism but the correctness argument becomes much harder... I can later think of speculative but initial design will be very simple cause correctness >> speed.

Correctness — how we know it is right

Three things guarantee correctness:

1. Causality — the dependency graph ensures no process runs before its inputs are ready. If A → B, A always finishes before B starts.

2. Delta cycle correctness...barrier after every delta ensures all processes in delta N complete before any process in delta N+1 begins. VHDL semantics preserved.

3. Determinism — signal updates are atomic (no partial writes visible to other threads). We hash the waveform trace of both sequential and parallel runs. Identical hash means identical simulation. Non-identical hash means a bug.

Performance evaluation plan

Test circuits I plan to work with for now:

- Basic logic gates
- 4-bit ripple carry adder
- 8-bit ALU
- Basic FSM
- Simple pipeline

Metrics:

- Speedup vs cores (1, 2, 4, 8)
- Scalability limits
- Synchronization overhead as % of total time

Expected result: Speedup scales with number of independent processes per timestamp. Circuits with more parallelism (like a wide ALU) should scale better than deeply sequential circuits (like a ripple carry adder).

Timeline

That is the plan. Sequential first, parallel second, correctness always.

A detailed idea of what i would do is here (full day wise plan on github): README on Github

Project Timeline Summary

phase 1 — research and foundation (days 1–20)

dates	what happened	status
13 feb to 15 feb	research and reading	done
16 feb – 25 feb	No Progress (Midsem exam prep)	done
25 feb	built event.h, event_queue.h	done
26 feb	built event_queue.c, tested main.c	done
27 feb	submission of design doc	done

design doc was written on 26–27 feb. parser was not started. sequential simulator not started. thats the honest picture... I do agree and understand my mistake of having done less in this phase.

phase 2 — sequential simulator (days 21–40)

days 21–25 cover signal and process representation, sensitivity list matching, and delta cycle engine. days 26–29 build the simulation loop with hardcoded test circuits (AND gate, flip flop) and VCD waveform output for GTKWave visualization. days 30–35 build the Flex/Bison parser for the VHDL subset and connect it to the simulator. days 36–40 are end-to-end testing on real VHDL circuits, bug fixing, and submission.

phase 3 — parallel implementation (days 41–70)

days 41–43 build the dependency graph — nodes are processes, edges are shared signals — to identify which processes can safely run in parallel at the same timestamp. days 44–48 implement the parallel execution engine using OpenMP with barriers at delta boundaries and atomic signal updates. days 49–56 focus entirely on correctness verification via trace hashing and performance measurement across 1, 2, and 4 cores. days 57–70 cover harder benchmarks (pipeline, 8-bit ALU), stress testing, scalability analysis, and final cleanup.

phase 4 — demo and benchmarks (days 71–80)

days 71–75 clean up waveform output, prepare demo circuits, and write the performance evaluation. days 76–80 cover final documentation, demo rehearsal, and submission.

key deliverables

day	deliverable
20	design document
40	sequential baseline simulator
70	parallel implementation
80	demo + GUI + benchmarks

---

sequential first. parallel second. correctness always.